

Name : Adhithya.S
Reg. No: 192412352

CODE:

```
i# Implementation of Expectation-Maximization (EM) Algorithm
```

```
import numpy as np
```

```
# Sample 1D dataset
```

```
X = np.array([1.0, 1.5, 2.0, 2.5, 5.0, 5.5, 6.0, 6.5])
```

```
# Number of Gaussian clusters
```

```
K = 2
```

```
# Number of data points
```

```
N = len(X)
```

```
# Initialize parameters
```

```
np.random.seed(42)
```

```
means = np.array([2.0, 6.0])
```

```
variances = np.array([1.0, 1.0])
```

```
weights = np.array([0.5, 0.5])
```

```
# Training parameters
```

```
iterations = 100
```

```
epsilon = 1e-6
```

```
# Gaussian Probability Density Function
```

```
def gaussian_pdf(x, mean, variance):
```

```
    return (1 / np.sqrt(2 * np.pi * variance)) * \
           np.exp(-((x - mean) ** 2) / (2 * variance))
```

```
# EM Algorithm
```

```
for iteration in range(iterations):
```

```
    # ----- E-STEP -----
```

```
    # Calculate responsibilities
```

```
    responsibilities = np.zeros((N, K))
```

```
    for i in range(N):
```

```
        for k in range(K):
```

```
            responsibilities[i, k] = (
                weights[k] *
                gaussian_pdf(X[i], means[k], variances[k])
            )
```

```
    # Normalize responsibilities
```

```
    responsibilities[i] = (
        responsibilities[i] /
        np.sum(responsibilities[i])
    )
```

```
    # ----- M-STEP -----
```

```
    # Update parameters
```

```
    Nk = np.sum(responsibilities, axis=0)
```

```
    # Update means
```

```

for k in range(K):
    means[k] = (
        np.sum(responsibilities[:, k] * X) /
        Nk[k]
    )

# Update variances
for k in range(K):
    variances[k] = (
        np.sum(
            responsibilities[:, k] *
            (X - means[k]) ** 2
        ) /
        Nk[k]
    )

# Update mixing weights
weights = Nk / N

# Display values every 10 iterations
if iteration % 10 == 0:
    print("\nIteration:", iteration + 1)
    print("Means:", means)
    print("Variances:", variances)
    print("Weights:", weights)

# ----- FINAL RESULTS -----

print("\n" + "=" * 40)
print("FINAL EM ALGORITHM RESULTS")
print("=" * 40)

print("\nFinal Means:")
print(means)

print("\nFinal Variances:")
print(variances)

print("\nFinal Mixing Weights:")
print(weights)

# Assign each data point to the cluster
clusters = np.argmax(responsibilities, axis=1)

print("\nData Point Clusters:")

for i in range(N):
    print(
        "Data:",
        X[i],
        "-> Cluster:",
        clusters[i] + 1
    )

```

OUTPUT :

```
Final Means:  
[1.75000007 5.74999993]
```

```
Final Variances:  
[0.31250028 0.31250028]
```

```
Final Mixing Weights:  
[0.5 0.5]
```

```
Data Point Clusters:  
Data: 1.0 -> Cluster: 1  
Data: 1.5 -> Cluster: 1  
Data: 2.0 -> Cluster: 1  
Data: 2.5 -> Cluster: 1  
Data: 5.0 -> Cluster: 2  
Data: 5.5 -> Cluster: 2  
Data: 6.0 -> Cluster: 2  
Data: 6.5 -> Cluster: 2
```